

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

Τμήμα Πληροφορικής



Τεχνικό εγχειρίδιο στην εργασία MVC μοντέλο
στο μάθημα
«ΣΥΓΧΡΟΝΑ ΘΕΜΑΤΑ ΤΕΧΝΟΛΟΓΙΑΣ ΛΟΓΙΣΜΙΚΟΥ-
ΛΟΓΙΣΜΙΚΟ ΓΙΑ ΚΙΝΗΤΕΣ ΣΥΣΚΕΥΕΣ»

Μάθημα	ΣΥΓΧΡΟΝΑ ΘΕΜΑΤΑ ΤΕΧΝΟΛΟΓΙΑΣ ΛΟΓΙΣΜΙΚΟΥ
Όνομα φοιτητών	Άγγελος Σακελλαρίου
Ημερομηνία παράδοσης	Πέμπτη 30 Ιανουαρίου 2025



Περιεχόμενα

Εισαγωγή	3
Σύνδεση με τη Βάση Δεδομένων	4
Μεθοδολογία Σύνδεσης	4
TelecomDBContext	6
Models	7
Client	7
ParcialClasses	8
ClientMetaData	8
Controllers	9
Index	9
Create	10
Views	11



Εισαγωγή

Το **TelecomApp** είναι μια εφαρμογή βασισμένη στην αρχιτεκτονική **MVC (Model-View-Controller)**. Στόχος της εφαρμογής είναι η διαχείριση τηλεπικοινωνιακών δεδομένων, όπως πελάτες, κλήσεις, λογαριασμοί, τηλέφωνα και προγράμματα. Το MVC χωρίζεται στα εξής βασικά μέρη:

- **Models:** Αντιπροσωπεύουν τα δεδομένα της εφαρμογής και τη δομή της βάσης δεδομένων.
- **Views:** Παρουσιάζουν τα δεδομένα στον χρήστη με φιλικό τρόπο.
- **Controllers:** Χειρίζονται την επιχειρηματική λογική και τη σύνδεση μεταξύ Models και Views.

Η βάση δεδομένων διαχειρίζεται μέσω του **Entity Framework Core**, ενώ οι Controllers αναλαμβάνουν την αλληλεπίδραση ανάμεσα στα δεδομένα και τις Views.



Σύνδεση με τη Βάση Δεδομένων

Μεθοδολογία Σύνδεσης

Ακολούθησα την παρακάτω διαδικασία για να ρυθμίσω τη σύνδεση με τη βάση δεδομένων και να δημιουργήσω τα Models:

1. Εγκατάσταση των απαραίτητων πακέτων μέσω NuGet:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
```

```
Install-Package Microsoft.EntityFrameworkCore.Tools
```

2. Χρήση της εντολής Scaffold-DbContext για τη δημιουργία των Models και του DbContext:

```
Scaffold-DbContext -f "Server=LAPTOP-  
FG8PGK26;Database=TelecomDB;Trusted_Connection=True;Trust Server Certificate=True"  
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models -Context "TelecomDBContext" -  
DataAnnotations
```

3. Προσθήκη της υπηρεσίας DbContext στο αρχείο Program.cs:

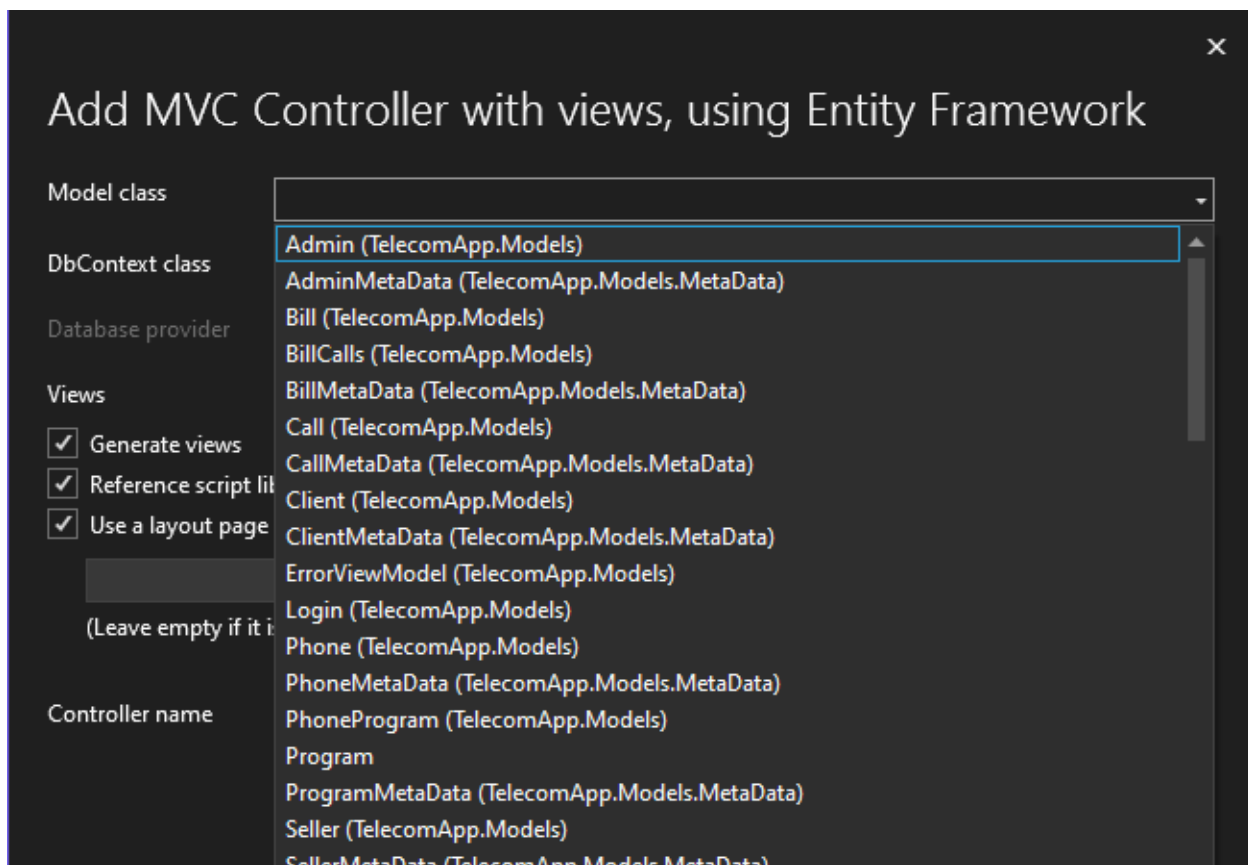
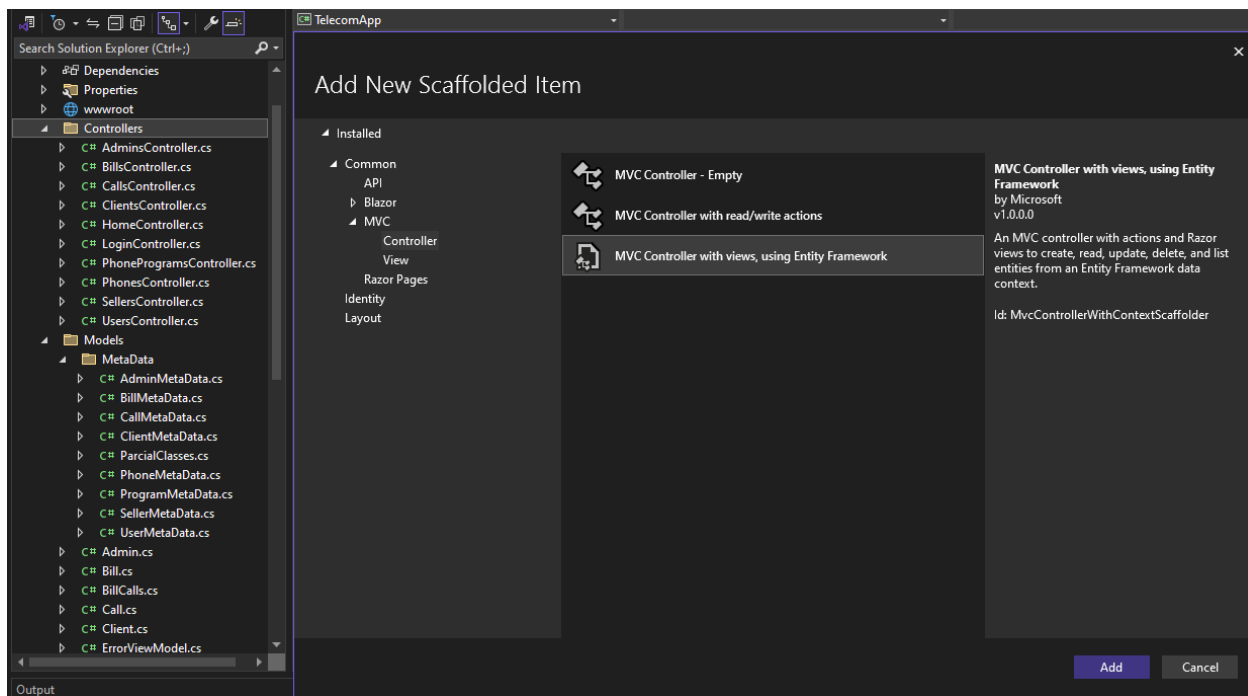
```
builder.Services.AddDbContext<TelecomDBContext>(options =>  
  
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

4. Διαμόρφωση του connection string στο αρχείο appsettings.Development.json:

```
"ConnectionStrings": {  
  
  "DefaultConnection": "Server=LAPTOP-  
FG8PGK26;Database=TelecomDB;Trusted_Connection=True;Trust Server  
Certificate=True;MultipleActiveResultSets=true"  
  
}
```

5. Δημιουργία των Controllers και Views μέσω scaffolding:

Χρησιμοποίησα τη δυνατότητα **"Add MVC Controller with Views using Entity Framework"** για τη δημιουργία των Controllers και Views. Αυτό μου επέτρεψε να συνδέσω τα δεδομένα με τις οθόνες της εφαρμογής αποτελεσματικά.





TelecomDBContext

Η κλάση **TelecomDBContext** λειτουργεί ως η κύρια γέφυρα μεταξύ της εφαρμογής C# και της βάσης δεδομένων TelecomDB, επιτρέποντας την απρόσκοπτη διαχείριση δεδομένων μέσω του Entity Framework Core. Ορίζει ιδιότητες DbSet για διάφορα tables, όπως Admins, Bills, Clients, Phones και άλλα, που αντιπροσωπεύουν το σχήμα της βάσης δεδομένων. Οι σχέσεις μεταξύ των οντοτήτων, όπως οι σχέσεις πολλών-προς-ένα και πολλών-προς-πολλά, ρυθμίζονται μέσω του Fluent API μέσα στη μέθοδο OnModelCreating. Επιπλέον, η κλάση περιλαμβάνει ρυθμίσεις για τη σύνδεση με τη βάση δεδομένων SQL Server. Υποστηρίζει επίσης dependency injection μέσω του παραμετρικού κατασκευαστή της, καθιστώντας την κατάλληλη για χρήση σε σύγχρονες εφαρμογές web ή desktop. Με την ενσωμάτωση της δομής και των σχέσεων της βάσης δεδομένων, η TelecomDBContext παρέχει μια ισχυρή βάση για την ερώτηση και τη διαχείριση των δεδομένων της TelecomDB.

```
1 using System;
2 using System.Collections.Generic;
3 using Microsoft.EntityFrameworkCore;
4
5 namespace TelecomApp.Models;
6
7 public partial class TelecomDBContext : DbContext
8 {
9     public TelecomDBContext()
10     {
11     }
12
13     public TelecomDBContext(DbContextOptions<TelecomDBContext> options)
14         : base(options)
15     {
16     }
17
18     public virtual DbSet<Admin> Admins { get; set; }
19
20     public virtual DbSet<Bill> Bills { get; set; }
21
22     public virtual DbSet<Call> Calls { get; set; }
23
24     public virtual DbSet<Client> Clients { get; set; }
25
26     public virtual DbSet<Phone> Phones { get; set; }
27
28     public virtual DbSet<PhoneProgram> PhonePrograms { get; set; }
29
30     public virtual DbSet<Seller> Sellers { get; set; }
31
32     public virtual DbSet<User> Users { get; set; }
33
34     protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
35     {
36         #warning To protect potentially sensitive information in your connection string, you should move it out of source code. You can avoid scaffolding
37         => optionsBuilder.UseSqlServer("Server=LAPTOP-FG8PGK26;Database=TelecomDB;Trusted_Connection=True;Trust Server Certificate=True");
38
39     }
40
41     protected override void OnModelCreating(ModelBuilder modelBuilder)
42     {
43         modelBuilder.Entity<Admin>(entity =>
44         {
45             entity.HasKey(e => e.AdminId).HasName("PK__admins__43AB7D29977D2BB4");
46             entity.HasOne(d => d.User).WithMany(p => p.Admins).HasConstraintName("FK__admins__user_id__02FC7413");
47         });
48
49         modelBuilder.Entity<Bill>(entity =>
50         {
51             entity.HasKey(e => e.BillId).HasName("PK__bills__D706D0B36C98E416");
52             entity.HasOne(d => d.Phone).WithMany(p => p.Bills).HasConstraintName("FK__bills__phone_id__05D8E0BE");
53             entity.HasMany(d => d.Calls).WithMany(p => p.Bills)
54                 .UsingEntity<Dictionary<string, object>>(>
55                 {
56                     "BillsCall",
57                     r => r.HasOne<Call>().WithMany()
58                         .HasForeignKey("CallId")
59                         .OnDelete(DeleteBehavior.Cascade)
60                         .HasConstraintName("FK__billsCall__call__0B91BA14"),
61                     l => l.HasOne<Bill>().WithMany()
62                         .HasForeignKey("BillId")
63                         .OnDelete(DeleteBehavior.Cascade)
64                         .HasConstraintName("FK__billsCall__bill__0A9D95DB"),
65                     j =>
66                 });
67         });
68     }
```

...



Models

Τα **Models** αντιπροσωπεύουν τα δεδομένα που χρησιμοποιούνται από την εφαρμογή.

Client

Η κλάση **Client** αντιπροσωπεύει τον πίνακα clients της βάσης δεδομένων και έχει διαμορφωθεί χρησιμοποιώντας το Entity Framework Core για τη χαρτογράφηση πινάκων και στηλών, καθώς και για τον ορισμό σχέσεων με άλλες οντότητες. Το πρωτεύον κλειδί είναι το ClientId, το οποίο αντιστοιχεί στη στήλη client_id, ενώ έχει επιπλέον ιδιότητες, όπως το Afm, το PhoneId και το UserId. Τα ξένα κλειδιά PhoneId και UserId συσχετίζονται με τους πίνακες Phone και User, αντίστοιχα, επιτρέποντας την εύκολη πρόσβαση σε σχετικά δεδομένα.

```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel.DataAnnotations;
4 using System.ComponentModel.DataAnnotations.Schema;
5 using Microsoft.EntityFrameworkCore;
6
7 namespace TelecomApp.Models;
8
9 [Table("clients")]
10 public partial class Client
11 {
12     [Key]
13     [Column("client_id")]
14     public int ClientId { get; set; }
15
16     [Column("AFM")]
17     [StringLength(50)]
18     public string? Afm { get; set; }
19
20     [Column("phone_id")]
21     public int? PhoneId { get; set; }
22
23     [Column("user_id")]
24     public int? UserId { get; set; }
25
26     [ForeignKey("PhoneId")]
27     [InverseProperty("Clients")]
28     public virtual Phone? Phone { get; set; }
29
30     [ForeignKey("UserId")]
31     [InverseProperty("Clients")]
32     public virtual User? User { get; set; }
33 }
```



PartialClasses

Η **PartialClasses** ορίζει κλάσεις μοντέλων και συσχετίζει κάθε μοντέλο με τη μεταδεδομένη κλάση του μέσω του χαρακτηριστικού `[ModelMetadataType]`. Οι κλάσεις μεταδεδομένων περιέχουν κανόνες επικύρωσης και ρυθμίσεις εμφάνισης, διαχωρίζοντας τις ευθύνες. Η κλάση `User` περιλαμβάνει την ιδιότητα `FullName`, που συνδυάζει δυναμικά τα πεδία `FirstName` και `LastName`, συνοδευόμενη από το χαρακτηριστικό `[Display]` για την εμφάνιση. Ο σχεδιασμός αυτός εξωτερικεύει τη λογική επικύρωσης και εμφάνισης, βελτιώνοντας τη συντηρησιμότητα.

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.Build.Evaluation;
3 using TelecomApp.Models.Metadata;
4 using System;
5 using System.ComponentModel.DataAnnotations;
6
7 namespace TelecomApp.Models
8 {
9     [ModelMetadataType(typeof(UserMetadata))]
10    public partial class User
11    {
12        [Display(Name = "Full Name")]
13        public string FullName => $"{FirstName} {LastName}";
14    }
15
16    [ModelMetadataType(typeof(AdminMetadata))]
17    public partial class Admin
18    {
19    }
20
21    [ModelMetadataType(typeof(SellerMetadata))]
22    public partial class Seller
23    {
24    }
25
26    [ModelMetadataType(typeof(ClientMetadata))]
27    public partial class Client
28    {
29    }
30
31    [ModelMetadataType(typeof(BillMetadata))]
32    public partial class Bill
33    {
34    }
35 }
```

...

ClientMetadata

Η κλάση **ClientMetadata** παρέχει μεταδεδομένα για το μοντέλο `Client`, κυρίως για εμφάνιση και επικύρωση. Μέσω χαρακτηριστικών όπως `[Display]`, καθορίζονται ονόματα εμφάνισης για ιδιότητες όπως `UserId` ("Username"), `User` ("Full Name") και `PhoneId` ("Phone"). Έτσι, το μοντέλο παραμένει καθαρό και η εμφάνιση διαχειρίζεται κεντρικά, διευκολύνοντας τη συντήρηση και εξασφαλίζοντας συνέπεια στη διεπαφή χρήστη.

```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace TelecomApp.Models.Metadata
4 {
5     public partial class ClientMetadata
6     {
7         [Display(Name = "Username")]
8         public int UserId { get; set; }
9
10        [Display(Name = "Full Name")]
11        public virtual User User { get; set; } = null!;
12
13        [Display(Name = "Phone")]
14        public int? PhoneId { get; set; }
15    }
16 }
```




Controllers

Τα **Controllers** χειρίζονται το business logic και την επικοινωνία με τα Views.

Index

Η ενέργεια **Index** ανακτά όλα τα αρχεία Client μαζί με τις συσχετισμένες οντότητες Phone και User. Στη συνέχεια, τα αντιστοιχεί σε μια λίστα αντικειμένων UserClient, τα οποία συνδυάζουν τις λεπτομέρειες των Client, Phone και User, και επιστρέφει τη λίστα στην προβολή.

```
21 // GET: Clients
22 3 references
23 public async Task<IActionResult> Index(string userType, string userId)
24 {
25     ViewData["UserType"] = userType;
26     ViewData["UserId"] = userId;
27
28     var clients = await _context.Clients
29         .Include(c => c.Phone)
30         .ThenInclude(p => p.Program)
31         .Include(c => c.User)
32         .ToListAsync();
33
34     var userClients = clients.Select(c => new UserClient
35     {
36         User = c.User,
37         Client = c,
38         Phone = c.Phone
39     }).ToList();
40
41     return View(userClients);
42 }
```



Create

Η ενέργεια **Create** προετοιμάζει την προβολή για τη δημιουργία ενός νέου Client, παρέχοντας μια λίστα επιλογών για το PhoneProgram (μέσω ενός SelectList).

```
73 // GET: Clients/Create
74 0 references
75 public IActionResult Create(string userType, string userId)
76 {
77     ViewData["UserType"] = userType;
78     ViewData["UserId"] = userId;
79     ViewData["ProgramId"] = new SelectList(_context.PhonePrograms, "ProgramId", "ProgramName");
80     return View();
}
```

Η ενέργεια **Create (POST)** λαμβάνει ένα μοντέλο UserClient, το οποίο περιλαμβάνει έναν νέο User, Phone και Client. Αρχικά προσθέτει τις οντότητες User και Phone στη βάση δεδομένων, ανακτά τα αντίστοιχα IDs τους και τα συσχετίζει με τη νέα οντότητα Client. Αφού αποθηκευτεί ο νέος Client, γίνεται ανακατεύθυνση στην ενέργεια Index.

```
82 // POST: Clients/Create
83 // To protect from overposting attacks, enable the specific properties you want to bind to.
84 // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
85 [HttpPost]
86 [ValidateAntiForgeryToken]
87 0 references
88 public async Task<IActionResult> Create(UserClient userClient, string userType, string userId)
89 {
90     if (ModelState.IsValid)
91     {
92         // Add User
93         _context.Users.Add(userClient.User);
94         await _context.SaveChangesAsync();
95
96         // Add Phone
97         _context.Phones.Add(userClient.Phone);
98         await _context.SaveChangesAsync();
99
100         // Get the inserted PhoneId
101         userClient.Client.PhoneId = userClient.Phone.PhoneId;
102
103         // Get the inserted UserId
104         userClient.Client.UserId = userClient.User.UserId;
105
106         // Add Client
107         _context.Clients.Add(userClient.Client);
108         await _context.SaveChangesAsync();
109
110         return RedirectToAction(nameof(Index), new { userType, userId });
111     }
112
113     ViewData["UserType"] = userType;
114     ViewData["UserId"] = userId;
115     ViewData["ProgramId"] = new SelectList(_context.PhonePrograms, "ProgramId", "ProgramName", userClient.Phone.ProgramId);
116     return View(userClient);
}
```



Views

Τα **Views** παρουσιάζουν τα δεδομένα στον χρήστη χρησιμοποιώντας Razor syntax.

Η προβολή **Index** εμφανίζει μια λίστα των αντικειμένων `UserClient` σε μορφή πίνακα. Ο πίνακας εμφανίζει πληροφορίες για κάθε πελάτη, όπως το όνομα, το επώνυμο, τον αριθμό τηλεφώνου και το όνομα προγράμματος του τηλεφώνου που είναι συσχετισμένο με τον πελάτη. Η προβολή χρησιμοποιεί το `@Html.DisplayNameFor()` για να δημιουργήσει τις επικεφαλίδες των στηλών, ενώ με το `@foreach` βρόχο προβάλλει κάθε εγγραφή στην αντίστοιχη γραμμή του πίνακα. Κάθε γραμμή περιλαμβάνει επίσης συνδέσμους για να εκτελούνται ενέργειες όπως η αλλαγή του προγράμματος του τηλεφώνου, η προσθήκη λογαριασμού, η προβολή κλήσεων και η προβολή λεπτομερειών του πελάτη. Οι σύνδεσμοι χρησιμοποιούν παραμέτρους διαδρομής για να μεταφέρουν δεδομένα (όπως `userType` και `userId`) στις ενέργειες του Controller. Τέλος, υπάρχει ένα κουμπί για τη δημιουργία νέου πελάτη, το οποίο κατευθύνει τον χρήστη στην αντίστοιχη ενέργεια δημιουργίας πελάτη.

```
@model IEnumerable<TelecomApp.Models.UserClient>

@{
    ViewData["Title"] = "Index";
    var userType = ViewData["UserType"].ToString();
    var userId = ViewData["UserId"].ToString();
}

<h1>Clients</h1>

<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.User.FirstName)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.User.LastName)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Phone.PhoneNumber)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Phone.ProgramId)
            </th>
            <th>
                Actions
            </th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model) {
            <tr>
                <td>
                    @Html.DisplayFor(modelItem => item.User.FirstName)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.User.LastName)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Phone.PhoneNumber)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Phone.Program.ProgramName)
                </td>
                <td>
                    <a asp-action="Edit" asp-controller="Phones" asp-route-id="@item.Phone.PhoneId" asp-route-userType="@userType" asp-route-userId="@userId" class="btn btn-dark">Change Program</a>
                    <a asp-action="Create" asp-controller="Bills" asp-route-phoneId="@item.Phone.PhoneId" asp-route-userType="@userType" asp-route-userId="@userId" class="btn btn-secondary">Add Bill</a>
                    <a asp-action="Index" asp-controller="Calls" asp-route-clientId="@item.User.UserId" asp-route-userType="@userType" asp-route-userId="@userId" class="btn btn-dark">View Calls</a>
                    <a asp-action="Details" asp-route-id="@item.Client.ClientId" asp-route-userType="@userType" asp-route-userId="@userId" class="btn btn-secondary">Details</a>
                </td>
            </tr>
        }
    </tbody>
</table>

<p>
    <a asp-action="Create" asp-route-userType="@userType" asp-route-userId="@userId" class="btn btn-dark">Create Client</a>
</p>
```